Recitation: Derivative Practice + Attention

Building Calculus Muscle for HW2, Modern Optimizers, and Attention Mechanisms

Timothy Obiso

Department of Computer Science
Brandeis University

COSI 115b — Spring 2026

Today's Plan

1. Derivative Practice with Loss Functions
2. Optimizers: From Adam to Muon
3. Dependency Parsing
4. Attention Mechanisms

Why Are We Doing This?

HW2 Part 1 asks you to take derivatives of the Word2Vec loss.

Instead of working through the HW problems, let's practice the *same skills* on **different** loss functions from modern architectures:

- Chain rule through log, exp, $\sum$, tanh
- Derivatives through softmax in non-obvious settings
- Matrix/vector calculus and the shape convention
- Interpreting gradients geometrically

If you can do these, you have more than enough technique for the HW.

Warm-up: L2 Regularization (Weight Decay)

Regularized MSE for $\hat{y} = \mathbf{w}^\top \mathbf{x}$:

$$J(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Warm-up: L2 Regularization (Weight Decay)

Regularized MSE for $\hat{y} = \mathbf{w}^\top \mathbf{x}$:

$$J(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Exercise: Compute $\dfrac{\partial J}{\partial \mathbf{w}}$.

Warm-up: L2 Regularization (Weight Decay)

Regularized MSE for $\hat{y} = \mathbf{w}^\top \mathbf{x}$:

$$J(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Exercise: Compute $\dfrac{\partial J}{\partial \mathbf{w}}$.

$$\frac{\partial J}{\partial \mathbf{w}} = (\hat{y} - y)\,\mathbf{x} + \lambda \mathbf{w}$$

Warm-up: L2 Regularization (Weight Decay)

Regularized MSE for $\hat{y} = \mathbf{w}^\top \mathbf{x}$:

$$J(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2 + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Exercise: Compute $\dfrac{\partial J}{\partial \mathbf{w}}$.

$$\frac{\partial J}{\partial \mathbf{w}} = (\hat{y} - y)\,\mathbf{x} + \lambda \mathbf{w}$$

In the SGD update: $\mathbf{w} \leftarrow (1 - \alpha\lambda)\mathbf{w} - \alpha(\hat{y} - y)\mathbf{x}$.

The $\lambda \mathbf{w}$ term multiplicatively shrinks weights each step — this is weight decay.

Shape check: $(\hat{y} - y)$ is scalar, $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{w} \in \mathbb{R}^d \Rightarrow$ result $\in \mathbb{R}^d$. ✓

Log-Sum-Exp: A Recurring Building Block

You'll encounter this structure throughout ML. For any:

$$f(\theta) = \log \sum_{j=1}^{K} \exp(g_j(\theta))$$

Log-Sum-Exp: A Recurring Building Block

You'll encounter this structure throughout ML. For any:

$$f(\theta) = \log \sum_{j=1}^{K} \exp(g_j(\theta))$$

By the chain rule:

$$\frac{\partial f}{\partial \theta} = \frac{\sum_j \exp(g_j(\theta)) \cdot \frac{\partial g_j}{\partial \theta}}{\sum_{j'} \exp(g_{j'}(\theta))} = \sum_j \underbrace{\frac{\exp(g_j)}{\sum_{j'} \exp(g_{j'})}}_{\text{softmax}_j} \cdot \frac{\partial g_j}{\partial \theta}$$

Log-Sum-Exp: A Recurring Building Block

You'll encounter this structure throughout ML. For any:

$$f(\theta) = \log \sum_{j=1}^{K} \exp(g_j(\theta))$$

By the chain rule:

$$\frac{\partial f}{\partial \theta} = \frac{\sum_j \exp(g_j(\theta)) \cdot \frac{\partial g_j}{\partial \theta}}{\sum_{j'} \exp(g_{j'}(\theta))} = \sum_j \underbrace{\frac{\exp(g_j)}{\sum_{j'} \exp(g_{j'})}}_{\text{softmax}_j} \cdot \frac{\partial g_j}{\partial \theta}$$

Key Insight

The derivative of log-sum-exp is an **expectation under the softmax distribution**. This shows up in cross-entropy, energy-based models, contrastive learning, attention, and the HW.

Exercise 1: RNN Hidden State Gradient

An Elman RNN computes:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

Suppose the loss at time t is $J_t = \frac{1}{2}\|\mathbf{h}_t - \mathbf{h}^*\|^2$ (matching a target).

Exercise 1: RNN Hidden State Gradient

An Elman RNN computes:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

Suppose the loss at time t is $J_t = \frac{1}{2}\|\mathbf{h}_t - \mathbf{h}^*\|^2$ (matching a target).

Exercise: Compute $\dfrac{\partial J_t}{\partial \mathbf{W}_x}$.

Exercise 1: RNN Hidden State Gradient

An Elman RNN computes:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

Suppose the loss at time t is $J_t = \frac{1}{2}\|\mathbf{h}_t - \mathbf{h}^*\|^2$ (matching a target).

Exercise: Compute $\dfrac{\partial J_t}{\partial \mathbf{W}_x}$.

Let $\mathbf{a}_t = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}$ (pre-activation). Then:

$$\frac{\partial J_t}{\partial \mathbf{h}_t} = \mathbf{h}_t - \mathbf{h}^*$$

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t} = \text{diag}(1 - \mathbf{h}_t^2) \quad \text{(element-wise } \tanh' \text{)}$$

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{W}_x} = ? \quad \text{(what shape should this be?)}$$

RNN Gradient: Completing the Chain

Define $\boldsymbol{\delta}_t = (\mathbf{h}_t - \mathbf{h}^*) \odot (1 - \mathbf{h}_t^2) \in \mathbb{R}^{d_h}$ (the "error signal").

RNN Gradient: Completing the Chain

Define $\boldsymbol{\delta}_t = (\mathbf{h}_t - \mathbf{h}^*) \odot (1 - \mathbf{h}_t^2) \in \mathbb{R}^{d_h}$ (the "error signal").

Since $\mathbf{a}_t = \mathbf{W}_x \mathbf{x}_t + \ldots$ and we need $\partial J / \partial \mathbf{W}_x \in \mathbb{R}^{d_h \times d_x}$:

$$\frac{\partial J_t}{\partial \mathbf{W}_x} = \boldsymbol{\delta}_t \, \mathbf{x}_t^\top \in \mathbb{R}^{d_h \times d_x} \quad \checkmark$$

RNN Gradient: Completing the Chain

Define $\boldsymbol{\delta}_t = (\mathbf{h}_t - \mathbf{h}^*) \odot (1 - \mathbf{h}_t^2) \in \mathbb{R}^{d_h}$ (the "error signal").

Since $\mathbf{a}_t = \mathbf{W}_x \mathbf{x}_t + \ldots$ and we need $\partial J / \partial \mathbf{W}_x \in \mathbb{R}^{d_h \times d_x}$:

$$\frac{\partial J_t}{\partial \mathbf{W}_x} = \boldsymbol{\delta}_t \, \mathbf{x}_t^\top \in \mathbb{R}^{d_h \times d_x} \quad \checkmark$$

Three skills at once

1. Chain rule through a nonlinearity (tanh)
2. Using diag$(\cdot)$ for element-wise derivative in matrix form
3. Outer product for the weight gradient (shape convention!)

RNN Gradient: Completing the Chain

Define $\boldsymbol{\delta}_t = (\mathbf{h}_t - \mathbf{h}^*) \odot (1 - \mathbf{h}_t^2) \in \mathbb{R}^{d_h}$ (the "error signal").

Since $\mathbf{a}_t = \mathbf{W}_x \mathbf{x}_t + \ldots$ and we need $\partial J / \partial \mathbf{W}_x \in \mathbb{R}^{d_h \times d_x}$:

$$\frac{\partial J_t}{\partial \mathbf{W}_x} = \boldsymbol{\delta}_t \, \mathbf{x}_t^\top \in \mathbb{R}^{d_h \times d_x} \quad \checkmark$$

Three skills at once

1. Chain rule through a nonlinearity (tanh)
2. Using diag$(\cdot)$ for element-wise derivative in matrix form
3. Outer product for the weight gradient (shape convention!)

Bonus: What about $\partial J_t / \partial \mathbf{W}_h$? The chain goes through $\mathbf{h}_{t-1}$, which itself depends on $\mathbf{W}_h$ — this is backpropagation through time.

Exercise 2: Attention Score Gradient

In scaled dot-product attention, the output for query $\mathbf{q}_i$ is:

$$\mathbf{o}_i = \sum_j \alpha_{ij}\, \mathbf{v}_j, \qquad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d})}$$

Exercise 2: Attention Score Gradient

In scaled dot-product attention, the output for query $\mathbf{q}_i$ is:

$$\mathbf{o}_i = \sum_j \alpha_{ij}\, \mathbf{v}_j, \qquad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d})}$$

Exercise: Compute $\dfrac{\partial \alpha_{ij}}{\partial \mathbf{q}_i}$ for a fixed i.

Exercise 2: Attention Score Gradient

In scaled dot-product attention, the output for query $\mathbf{q}_i$ is:

$$\mathbf{o}_i = \sum_j \alpha_{ij}\, \mathbf{v}_j, \qquad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d})}$$

Exercise: Compute $\dfrac{\partial \alpha_{ij}}{\partial \mathbf{q}_i}$ for a fixed i.

This is the **softmax Jacobian** applied to attention. Note that α_{ij} depends on $\mathbf{q}_i^\top \mathbf{k}_j$ for *all* j (through the denominator), so:

$$\frac{\partial \alpha_{ij}}{\partial \mathbf{q}_i} = \, ?$$

Hint: start from $\alpha_{ij} = \text{softmax}_j(\mathbf{q}_i^\top \mathbf{K} / \sqrt{d})$ and use the chain rule.

Attention Score Gradient: Solution

Let $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}$. The softmax Jacobian gives:

$$\frac{\partial \alpha_{ij}}{\partial s_{il}} = \alpha_{ij}(\mathbf{1}[j = l] - \alpha_{il})$$

Attention Score Gradient: Solution

Let $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}$. The softmax Jacobian gives:

$$\frac{\partial \alpha_{ij}}{\partial s_{il}} = \alpha_{ij}(\mathbf{1}[j = l] - \alpha_{il})$$

Then by chain rule ($\partial s_{il} / \partial \mathbf{q}_i = \mathbf{k}_l / \sqrt{d}$):

$$\frac{\partial \alpha_{ij}}{\partial \mathbf{q}_i} = \sum_l \alpha_{ij}(\mathbf{1}[j = l] - \alpha_{il}) \cdot \frac{\mathbf{k}_l}{\sqrt{d}} = \frac{\alpha_{ij}}{\sqrt{d}} \left(\mathbf{k}_j - \sum_l \alpha_{il} \mathbf{k}_l \right)$$

Attention Score Gradient: Solution

Let $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}$. The softmax Jacobian gives:

$$\frac{\partial \alpha_{ij}}{\partial s_{il}} = \alpha_{ij}(\mathbf{1}[j = l] - \alpha_{il})$$

Then by chain rule ($\partial s_{il} / \partial \mathbf{q}_i = \mathbf{k}_l / \sqrt{d}$):

$$\frac{\partial \alpha_{ij}}{\partial \mathbf{q}_i} = \sum_l \alpha_{ij}(\mathbf{1}[j = l] - \alpha_{il}) \cdot \frac{\mathbf{k}_l}{\sqrt{d}} = \frac{\alpha_{ij}}{\sqrt{d}} \left(\mathbf{k}_j - \sum_l \alpha_{il} \mathbf{k}_l \right)$$

Interpretation

The gradient is proportional to $\mathbf{k}_j - \bar{\mathbf{k}}_i$ where $\bar{\mathbf{k}}_i = \sum_l \alpha_{il} \mathbf{k}_l$ is the **expected key**. Increasing α_{ij} pushes $\mathbf{q}_i$ toward $\mathbf{k}_j$ and away from the attention-weighted average of all keys.

Compare this structure to the HW.

Exercise 3: Contrastive Loss (InfoNCE)

Used in CLIP, SimCLR, and many retrieval models. Given a positive pair (i, i^+) among K negatives:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_{i^+})/\tau)}{\sum_{k=1}^{K} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}/(\|\mathbf{a}\|\|\mathbf{b}\|)$ and τ is a temperature.

Exercise 3: Contrastive Loss (InfoNCE)

Used in CLIP, SimCLR, and many retrieval models. Given a positive pair (i, i^+) among K negatives:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_{i^+})/\tau)}{\sum_{k=1}^{K} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}/(\|\mathbf{a}\|\|\mathbf{b}\|)$ and τ is a temperature.

Exercise: Assuming normalized embeddings ($\|\mathbf{z}\| = 1$), simplify and compute $\dfrac{\partial \mathcal{L}_i}{\partial \mathbf{z}_i}$.

Exercise 3: Contrastive Loss (InfoNCE)

Used in CLIP, SimCLR, and many retrieval models. Given a positive pair (i, i^+) among K negatives:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_{i^+})/\tau)}{\sum_{k=1}^{K} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}/(\|\mathbf{a}\|\|\mathbf{b}\|)$ and τ is a temperature.

Exercise: Assuming normalized embeddings ($\|\mathbf{z}\| = 1$), simplify and compute $\dfrac{\partial \mathcal{L}_i}{\partial \mathbf{z}_i}$.

With normalized embeddings: $\text{sim} = \mathbf{z}_i^\top \mathbf{z}_k$, so:

$$\mathcal{L}_i = -\frac{\mathbf{z}_i^\top \mathbf{z}_{i^+}}{\tau} + \log \sum_k \exp\left(\frac{\mathbf{z}_i^\top \mathbf{z}_k}{\tau}\right)$$

What technique do you use to differentiate this?

Contrastive Loss Gradient: Solution

Split the log, apply our log-sum-exp result:

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{z}_i} = -\frac{\mathbf{z}_{i^+}}{\tau} + \sum_k \frac{\exp(\mathbf{z}_i^\top \mathbf{z}_k/\tau)}{\sum_{k'} \exp(\mathbf{z}_i^\top \mathbf{z}_{k'}/\tau)} \cdot \frac{\mathbf{z}_k}{\tau} = \frac{1}{\tau}\left(-\mathbf{z}_{i^+} + \sum_k p_k \, \mathbf{z}_k\right)$$

where $p_k = \text{softmax}_k(\mathbf{z}_i^\top \mathbf{z}_k/\tau)$.

Contrastive Loss Gradient: Solution

Split the log, apply our log-sum-exp result:

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{z}_i} = -\frac{\mathbf{z}_{i^+}}{\tau} + \sum_k \frac{\exp(\mathbf{z}_i^\top \mathbf{z}_k/\tau)}{\sum_{k'} \exp(\mathbf{z}_i^\top \mathbf{z}_{k'}/\tau)} \cdot \frac{\mathbf{z}_k}{\tau} = \frac{1}{\tau}\left(-\mathbf{z}_{i^+} + \sum_k p_k \, \mathbf{z}_k\right)$$

where $p_k = \text{softmax}_k(\mathbf{z}_i^\top \mathbf{z}_k/\tau)$.

Interpretation

- $-\mathbf{z}_{i^+}$: pull $\mathbf{z}_i$ toward the positive example
- $+\sum_k p_k \mathbf{z}_k$: push $\mathbf{z}_i$ away from the expected negative (weighted by current similarity)

Contrastive Loss Gradient: Solution

Split the log, apply our log-sum-exp result:

$$\frac{\partial \mathcal{L}_i}{\partial \mathbf{z}_i} = -\frac{\mathbf{z}_{i^+}}{\tau} + \sum_k \frac{\exp(\mathbf{z}_i^\top \mathbf{z}_k/\tau)}{\sum_{k'} \exp(\mathbf{z}_i^\top \mathbf{z}_{k'}/\tau)} \cdot \frac{\mathbf{z}_k}{\tau} = \frac{1}{\tau}\left(-\mathbf{z}_{i^+} + \sum_k p_k \, \mathbf{z}_k\right)$$

where $p_k = \text{softmax}_k(\mathbf{z}_i^\top \mathbf{z}_k/\tau)$.

Interpretation

- $-\mathbf{z}_{i^+}$: pull $\mathbf{z}_i$ toward the positive example
- $+\sum_k p_k \mathbf{z}_k$: push $\mathbf{z}_i$ away from the expected negative (weighted by current similarity)

Role of temperature: Small τ makes p_k peakier (hard negatives dominate); large τ spreads the push across all negatives. The gradient structure is "target minus expectation" — a pattern worth recognizing.

Adam: The Equations

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla J \qquad \text{(momentum)}$$

$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla J \odot \nabla J) \qquad \text{(squared gradient EMA)}$$

$$\theta_{t+1} \leftarrow \theta_t - \alpha\, \mathbf{m}_{t+1} \,/\, \sqrt{\mathbf{v}_{t+1}} \qquad \text{(update)}$$

Adam: The Equations

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla J \qquad \text{(momentum)}$$

$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla J \odot \nabla J) \qquad \text{(squared gradient EMA)}$$

$$\theta_{t+1} \leftarrow \theta_t - \alpha\, \mathbf{m}_{t+1} \,/\, \sqrt{\mathbf{v}_{t+1}} \qquad \text{(update)}$$

Discussion questions (think about these for the HW):

1. Why does averaging gradients over time reduce variance?
2. $\mathbf{v}$ is large when gradients are consistently large. We divide by $\sqrt{\mathbf{v}}$. What effect does that have on frequent vs. rare features?
3. Adam treats each parameter independently. What information might that miss?

Adam: The Equations

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1)\nabla J \qquad \text{(momentum)}$$

$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla J \odot \nabla J) \qquad \text{(squared gradient EMA)}$$

$$\theta_{t+1} \leftarrow \theta_t - \alpha\, \mathbf{m}_{t+1} \,/\, \sqrt{\mathbf{v}_{t+1}} \qquad \text{(update)}$$

Discussion questions (think about these for the HW):

1. Why does averaging gradients over time reduce variance?
2. $\mathbf{v}$ is large when gradients are consistently large. We divide by $\sqrt{\mathbf{v}}$. What effect does that have on frequent vs. rare features?
3. Adam treats each parameter independently. What information might that miss?

That last question leads us to something new...

Adam's Limitation: Per-Parameter Updates

Adam computes a separate learning rate for each scalar parameter.

But in a transformer, parameters form **weight matrices $\mathbf{W} \in \mathbb{R}^{m \times n}$** that define linear transformations. The rows and columns have *structural relationships* that Adam ignores.

Adam's Limitation: Per-Parameter Updates

Adam computes a separate learning rate for each scalar parameter.

But in a transformer, parameters form **weight matrices $\mathbf{W} \in \mathbb{R}^{m \times n}$** that define linear transformations. The rows and columns have *structural relationships* that Adam ignores.

Analogy

Imagine adjusting a rotation matrix one entry at a time — you'd break the rotation property. Matrix-aware optimizers try to preserve the geometric structure of weight updates.

Muon: MomentUm Orthogonalized by Newton-Schulz

Muon (Jordan et al., 2024) treats gradient updates as *matrices* instead of flat vectors.

The core idea:

1. Compute momentum $\mathbf{M}_t$ of the gradient matrix (just like Adam's $\mathbf{m}$)
2. Orthogonalize $\mathbf{M}_t$ to get the nearest orthogonal matrix
3. Use the orthogonalized matrix as the update

Muon: MomentUm Orthogonalized by Newton-Schulz

Muon (Jordan et al., 2024) treats gradient updates as *matrices* instead of flat vectors.

The core idea:

1. Compute momentum $\mathbf{M}_t$ of the gradient matrix (just like Adam's $\mathbf{m}$)
2. Orthogonalize $\mathbf{M}_t$ to get the nearest orthogonal matrix
3. Use the orthogonalized matrix as the update

Update Rule

$$\mathbf{M}_{t+1} \leftarrow \mu \mathbf{M}_t + (1-\mu)\nabla_{\mathbf{W}} J$$

$$\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \eta \cdot \text{ortho}(\mathbf{M}_{t+1})$$

where $\text{ortho}(\mathbf{M})$ finds the nearest orthogonal matrix to $\mathbf{M}$.

What Does Orthogonalization Do?

For a matrix $\mathbf{M}$ with SVD $\mathbf{M} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$:

$$\text{ortho}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top$$

What Does Orthogonalization Do?

For a matrix $\mathbf{M}$ with SVD $\mathbf{M} = \mathbf{USV}^\top$:

$$\text{ortho}(\mathbf{M}) = \mathbf{UV}^\top$$

This **keeps the directions** ($\mathbf{U}$, $\mathbf{V}$) but **removes the magnitudes** ($\mathbf{S}$).

What Does Orthogonalization Do?

For a matrix $\mathbf{M}$ with SVD $\mathbf{M} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$:

$$\text{ortho}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top$$

This **keeps the directions** ($\mathbf{U}$, $\mathbf{V}$) but **removes the magnitudes** ($\mathbf{S}$).

Why is this helpful?

- All singular values become 1 — the update is **isotropic**
- No direction dominates the update; the optimizer explores all directions equally
- This is like Adam's adaptive learning rate but for *entire matrix directions*, not individual parameters

What Does Orthogonalization Do?

For a matrix $\mathbf{M}$ with SVD $\mathbf{M} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$:

$$\text{ortho}(\mathbf{M}) = \mathbf{U}\mathbf{V}^\top$$

This **keeps the directions** ($\mathbf{U}$, $\mathbf{V}$) but **removes the magnitudes** ($\mathbf{S}$).

Why is this helpful?

- All singular values become 1 — the update is **isotropic**
- No direction dominates the update; the optimizer explores all directions equally
- This is like Adam's adaptive learning rate but for *entire matrix directions*, not individual parameters

Connection to weight decay: L2 regularization also controls singular values, but bluntly (shrinks all weights). Muon's orthogonalization is more surgical.

Newton-Schulz: Fast Approximate Orthogonalization

Computing the full SVD is expensive ($O(mn\min(m,n))$). Muon uses **Newton-Schulz iteration**:

$$\mathbf{X}_0 = \mathbf{M}/\|\mathbf{M}\|_F$$

$$\mathbf{X}_{k+1} = a\mathbf{X}_k + b(\mathbf{X}_k\mathbf{X}_k^\top)\mathbf{X}_k + c(\mathbf{X}_k\mathbf{X}_k^\top)^2\mathbf{X}_k$$

Newton-Schulz: Fast Approximate Orthogonalization

Computing the full SVD is expensive ($O(mn\min(m,n))$). Muon uses **Newton-Schulz iteration**:

$$\mathbf{X}_0 = \mathbf{M}/\|\mathbf{M}\|_F$$
$$\mathbf{X}_{k+1} = a\mathbf{X}_k + b(\mathbf{X}_k\mathbf{X}_k^\top)\mathbf{X}_k + c(\mathbf{X}_k\mathbf{X}_k^\top)^2\mathbf{X}_k$$

After just $\sim$5 iterations, $\mathbf{X}_k \approx \mathbf{U}\mathbf{V}^\top$.

Why not just use SVD?

- Newton-Schulz uses only matrix multiplications — perfect for GPUs
- Runs stably in `bfloat16` (SVD needs `float32`)
- $< 1\%$ compute overhead vs. the forward/backward pass

Muon in Practice

Where Muon applies

- Only for **2D weight matrices** (attention projections, MLP layers)
- Embedding layers and output heads still use AdamW
- Scalars and biases still use AdamW

Muon in Practice

Where Muon applies

- Only for **2D weight matrices** (attention projections, MLP layers)
- Embedding layers and output heads still use AdamW
- Scalars and biases still use AdamW

Results

- $\sim$35% training speed improvement on NanoGPT benchmarks
- $\sim 2\times$ compute efficiency vs. AdamW at scale (Moonlight 3B/16B MoE)
- Uses only 1 momentum buffer (vs. Adam's 2) $\Rightarrow$ 33% less optimizer memory

Muon in Practice

Where Muon applies

- Only for **2D weight matrices** (attention projections, MLP layers)
- Embedding layers and output heads still use AdamW
- Scalars and biases still use AdamW

Results

- $\sim$35% training speed improvement on NanoGPT benchmarks
- $\sim 2\times$ compute efficiency vs. AdamW at scale (Moonlight 3B/16B MoE)
- Uses only 1 momentum buffer (vs. Adam's 2) $\Rightarrow$ 33% less optimizer memory

The takeaway: treating weight matrices as *geometric objects* rather than bags of numbers unlocks significant gains.

Dropout: Conceptual Questions for HW2

During training, randomly zero out hidden units:

$$\mathbf{h}_{\text{drop}} = \gamma\, \mathbf{d} \odot \mathbf{h}, \qquad d_i \sim \text{Bernoulli}(1 - p_{\text{drop}})$$

We want $\mathbb{E}[\mathbf{h}_{\text{drop}}]_i = h_i$, which determines γ.

Dropout: Conceptual Questions for HW2

During training, randomly zero out hidden units:

$$\mathbf{h}_{\text{drop}} = \gamma\, \mathbf{d} \odot \mathbf{h}, \qquad d_i \sim \text{Bernoulli}(1 - p_{\text{drop}})$$

We want $\mathbb{E}[\mathbf{h}_{\text{drop}}]_i = h_i$, which determines γ.

For the HW, think about:

- What does "co-adaptation" of neurons mean? Why is it bad?
- How does randomly disabling neurons during training prevent it?
- At test time, we want the model's *best* prediction. Why would randomness hurt here?
- What role does γ play in making training and test time compatible?

Transition-Based Parsing: The Mechanics

Maintain a **partial parse**: (stack, buffer, dependencies).

Three transitions:

1. **SHIFT**: Move first word from buffer $\rightarrow$ top of stack
2. **LEFT-ARC**: Top of stack is head of second item; remove second item
   - Adds dependency: first $\rightarrow$ second
3. **RIGHT-ARC**: Second item is head of top; remove top
   - Adds dependency: second $\rightarrow$ first

Terminate when buffer is empty and stack has size 1 (just ROOT).

Neural Network: Architecture Overview

Forward Pass

$$\mathbf{x} = [\mathbf{E}_{w_1}, \ldots, \mathbf{E}_{w_m}] \in \mathbb{R}^{dm}$$
$$\mathbf{h} = \text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)$$
$$\mathbf{l} = \mathbf{h}\mathbf{U} + \mathbf{b}_2$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{l})$$

Critical constraints:

- You must use `nn.Parameter` — **not** `nn.Linear` or `nn.Embedding`
- Work backwards from $\hat{\mathbf{y}} \in \mathbb{R}^3$ to figure out dimensions of each weight/bias
- For embedding lookup: PyTorch tensor indexing handles batches automatically — no for loops needed

From Word2Vec to Attention

Word2Vec

$\mathbf{u}_o^\top \mathbf{v}_c$ — a dot product measuring compatibility between *static* word vectors. "Banking" always has the same $\mathbf{v}_c$ regardless of context.

From Word2Vec to Attention

Word2Vec

$\mathbf{u}_o^\top \mathbf{v}_c$ — a dot product measuring compatibility between *static* word vectors. "Banking" always has the same $\mathbf{v}_c$ regardless of context.

Attention

Also uses dot products for compatibility — but between *context-dependent* representations. The same word gets different representations depending on surrounding words.

Both share the core idea: dot products measure relevance between vectors.

Self-Attention: Queries, Keys, Values

Given input embeddings $\mathbf{X} \in \mathbb{R}^{n \times d}$ (one row per token):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

Self-Attention: Queries, Keys, Values

Given input embeddings $\mathbf{X} \in \mathbb{R}^{n \times d}$ (one row per token):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}$$

Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V}$$

Self-Attention: Queries, Keys, Values

Given input embeddings $\mathbf{X} \in \mathbb{R}^{n \times d}$ (one row per token):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Break it down:

- $\mathbf{Q}\mathbf{K}^\top$: pairwise relevance scores ("How much should token i attend to token j?")
- $\sqrt{d_k}$: prevents dot products from getting too large (keeps softmax gradients healthy)
- Softmax (row-wise): normalize into attention weights
- Multiply by $\mathbf{V}$: output = weighted sum of value vectors

Worked Example: 3-Token Self-Attention

Suppose tokens $[t_1, t_2, t_3]$ with $d_k = 2$.

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} = \begin{pmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{pmatrix}$$

Worked Example: 3-Token Self-Attention

Suppose tokens $[t_1, t_2, t_3]$ with $d_k = 2$.

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} = \begin{pmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{pmatrix}$$

After row-wise softmax, we get attention weights $\mathbf{A}$ where each row sums to 1:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

Worked Example: 3-Token Self-Attention

Suppose tokens $[t_1, t_2, t_3]$ with $d_k = 2$.

$$\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} = \begin{pmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{pmatrix}$$

After row-wise softmax, we get attention weights $\mathbf{A}$ where each row sums to 1:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

Output for token 1: $a_{11}\,\mathbf{v}_1 + a_{12}\,\mathbf{v}_2 + a_{13}\,\mathbf{v}_3$

Each token's new representation is a weighted blend of all value vectors, weighted by relevance.

Multi-Head Attention

Run h attention heads in parallel with different learned projections:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q,\ \mathbf{X}\mathbf{W}_i^K,\ \mathbf{X}\mathbf{W}_i^V)$$

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O$$

Multi-Head Attention

Run h attention heads in parallel with different learned projections:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q,\ \mathbf{X}\mathbf{W}_i^K,\ \mathbf{X}\mathbf{W}_i^V)$$

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O$$

Why multiple heads?

- Different heads learn different types of relationships (syntactic, semantic, positional)
- Similar intuition to multiple convolutional filters
- Typically $d_k = d_v = d_{\text{model}}/h$, so total compute is similar to single-head

The Transformer Block

Each layer of a Transformer repeats:

1. **Multi-Head Self-Attention** + residual connection + LayerNorm

$$\mathbf{X}' = \text{LayerNorm}(\mathbf{X} + \text{MultiHead}(\mathbf{X}))$$

2. **Feed-Forward Network** + residual connection + LayerNorm

$$\mathbf{X}'' = \text{LayerNorm}(\mathbf{X}' + \text{FFN}(\mathbf{X}'))$$

Residual Connections

Adding the input back ($\mathbf{X} + \ldots$) helps gradients flow through deep networks — same principle as ResNets.

Causal (Decoder) Attention for LLMs

For autoregressive language models, token i can only attend to tokens $\leq i$:

$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \text{mask}\right)$$

Causal (Decoder) Attention for LLMs

For autoregressive language models, token i can only attend to tokens $\leq i$:

$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \text{mask}\right)$$

- Prevents "cheating" by looking at future tokens during generation
- This is the attention pattern used in GPT-style models